LLM-Inspired Methods for Temperature Prediction: Summary and Performance at 3-Hour Lead Time

Rubin Observatory Thermal Control Project

January 26, 2026

1 Overview

We explored several approaches inspired by Large Language Model (LLM) architectures for predicting sunset temperature at the Rubin Observatory site. These methods treat temperature time series as sequences analogous to text, applying embedding and attention-based techniques originally developed for natural language processing.

2 Methods

2.1 Temperature Tokenization

Two tokenization strategies were implemented:

2.1.1 Discrete Bin Tokenization

Temperature values are discretized into 100 bins spanning the observed range (∼-5°C to +20°C). Each bin becomes a "token" in a vocabulary, similar to how words are tokenized in language models.

- Vocabulary size: 100 temperature tokens + 3 special tokens (PAD, SEP, CLS)
- Bin width: ∼0.25°C

2.1.2 ASCII String Tokenization

Temperatures are converted to ASCII strings with 0.1°C precision (e.g., "+12.3", "-5.7"). This preserves the numerical structure while treating temperatures as "words."

- Vocabulary size: ∼250 unique temperature strings
- Precision: 0.1°C

2.2 Sequence Models

2.2.1 Embedding + LSTM

- 32-dimensional learned embeddings for each temperature token
- 2-layer bidirectional LSTM with 64/128 hidden units
- Attention mechanism for sequence summarization
- Final prediction: current temperature + learned ΔT

2.2.2 Embedding + Transformer

- 32/64-dimensional embeddings with learned positional encoding
- 2–3 layer transformer encoder with 4 attention heads
- Mean pooling or final-position summary (analogous to [CLS] token)
- Predicts temperature change from current to sunset

2.3 Training Details

- Input: 6 hours of temperature history (24 readings at 15-min intervals) or 3 days of context
- Train/test split: odd calendar days for training, even days for testing
- 15% of training data reserved for validation/early stopping
- Optimizer: Adam with learning rate 0.001, ReduceLROnPlateau scheduler
- Training epochs: 100 with early stopping

3 Performance at 3-Hour Lead Time

Table 1 summarizes prediction performance for sunset temperature when predictions are made 3 hours before sunset.

Table 1: Sunset Temperature Prediction Performance (3h Lead Time)

Method	**RMS (°C)**	**Bias (°C)**	**% <1°C Error**
Baseline Methods			
Persistence (current temp)	2.52	+2.24	—
Traditional ML Methods			
Linear Regression	1.07	+0.10	70%
Random Forest	1.09	+0.04	70%
Gradient Boosting	1.10	+0.01	69%
LLM-Inspired Methods			
ASCII Word LSTM (0.1°C precision)	1.28	+0.35	∼55%
ASCII Word LSTM (0.25°C precision)	1.36	+0.39	∼50%

4 Key Findings

4.1 Performance Comparison

- LLM-inspired methods achieve ∼1.28–1.36°C RMS at 3h lead time
- Traditional ML methods perform better: Random Forest achieves 1.09°C RMS
- All methods improve substantially over persistence baseline (2.52°C RMS)

- LLM methods show ∼50% improvement over persistence, but are ∼20% worse than Random Forest

4.2 Embedding Analysis

The learned temperature embeddings show meaningful structure:

- Adjacent temperature bins have high cosine similarity (∼0.8–0.9)
- PCA visualization shows smooth ordering by temperature value
- The model learns that nearby temperatures are semantically similar

4.3 Lead Time Dependence

Performance improves at shorter lead times. Reference values from traditional ML:

Lead Time	Random Forest RMS	Linear Regression RMS
3 hours	1.09°C	1.07°C
2 hours	1.01°C	0.94°C
1 hour	0.80°C	0.71°C

LLM methods follow similar trends but with ∼20% higher RMS at each lead time.

5 Tokenization Precision Comparison

We compared two tokenization precisions for the ASCII string approach:

Table 2: Effect of Tokenization Precision on Prediction Performance (3h Lead Time)

Metric	**0.1°C**	**0.25°C**	**Difference**
Vocabulary size	323 words	133 words	+190
RMS Error	1.282°C	1.359°C	−0.077°C
MAE	1.003°C	1.087°C	−0.085°C
Bias	+0.353°C	+0.387°C	−0.034°C

5.1 Key Observations

- **Finer precision helps modestly**: 0.1°C precision reduces RMS by 0.077°C compared to 0.25°C (approximately 6% relative improvement).
- **Vocabulary trade-off**: Finer precision requires 2.4× larger vocabulary (323 vs 133 words), increasing model complexity and training data requirements.
- **Diminishing returns**: The 0.077°C improvement is small compared to the ∼1.3°C total error, indicating that tokenization precision is not the limiting factor.
- **Practical recommendation**: 0.1°C precision provides a reasonable balance between granularity and vocabulary size. Going coarser than 0.25°C would likely degrade performance more significantly.

6 Conclusions

1. **LLM-inspired approaches work** for temperature prediction, achieving RMS errors of ∼1.28°C at 3-hour lead time with 0.1°C tokenization precision.

2. **Traditional ML methods perform better**: Random Forest achieves 1.09°C RMS—about 20% lower error than LLM methods with much simpler implementation.

3. **Learned embeddings are interpretable**: the model discovers that temperatures form an ordered, continuous space.

4. **Fundamental limits**: Weather unpredictability constrains improvement potential. The ∼1°C RMS floor for traditional ML appears to be a physical limit for 3-hour predictions at this site.

5. **Practical recommendation**: For operational use, Random Forest or Linear Regression provide better accuracy with lower computational cost and simpler deployment. LLM-inspired methods offer no advantage for this structured regression task.

7 Implementation Notes

Code is available in:

- `llm_embedding_prediction.py` – Discrete bin tokenization with LSTM/Transformer
- `ascii_llm_expanded.py` – ASCII string tokenization with multiple lead times
- `ascii_temperature_llm.py` – Original ASCII approach